

GYM-GAZEBO-SIM: A TOOLKIT FOR EXTENDING THE GYMNASIUM API FOR GAZEBO-SIM AND ROS2

T. Momose, M. Isbasescu*

Boye Ahlborn Engineering Physics Project Lab, University of British Columbia, Vancouver, Canada

Abstract

This white paper introduces **Gym-Gazebo-Sim** an extension to the Farama Foundation’s Gymnasium API for training reinforcement learning agents using **Gazebo Harmonic** and **ROS2 Jazzy**. It was designed for classroom use with a focus on being able to run at a real time factor of at least one on non-gaming laptops. The content discusses the software architecture of this extension and the results obtained using it on training reinforcement learning agents across two different environments: line following using **Q-Learning** and cart pole using **Cross-Entropy Learning**.

Gym-Gazebo-Sim code: [GitHub/ENPH353/gym-gazebo-sim](https://github.com/ENPH353/gym-gazebo-sim).

INTRODUCTION

Reinforcement learning (RL) has become essential in robotics for solving complex behaviors, driving the need for frameworks that smoothly bridge machine learning algorithms with high-fidelity simulators.

The Farama Foundation’s Gymnasium [1] is one of the standard RL environment frameworks, but integrating it with Gazebo Sim [2] - a popular open source simulator - requires substantial engineering overhead. This paper presents the **Gym-Gazebo-Sim** toolkit that extends the Gymnasium API for Gazebo Sim Harmonic to close this gap.

This toolkit was developed for use in ENPH 353 - an Engineering Physics introduction to software for robotics at the University of British Columbia. Students in this course often lack the robust hardware required to run high-fidelity simulators like NVIDIA Isaac Sim or MuJoCo. By using ROS2, Gazebo Sim and simple environments the Gym-Gazebo-Sim toolkit enables almost all the students in the class to practice hands-on reinforcement learning for robotics on their own computers.

The toolkit abstracts Gazebo’s communication infrastructure and introduces a plugin for resetting agents, allowing users to easily build RL environments. This paper will review the background of the Gazebo RL community, the proposed software architecture, and validate the toolkit’s performance.

BACKGROUND

The concept of using virtual simulation to train RL agents dates back to the late 1960s. One of the most influential

examples is the inverted pendulum (cart-pole) experiment conducted by Donald Michie and Roger Chambers in 1968 [3]. They used a computer simulation to train an RL controller known as BOXES to balance a pole by receiving a negative reward if the pole exceeded an angle-threshold or if the cart hit the end of its track.

Today, sim-to-real deployment is a critical component in robotics, offering major advantages in cost, safety, and efficiency. Simulations accelerate training through rapid resets and randomized noise while de-risking development by removing the need for physical prototypes.

Previous versions of the Gazebo simulator benefited from community-driven toolkits that bridged OpenAI’s Gym RL framework [4] (deprecated) with Gazebo via ROS. Prominent examples include Gym-Gazebo by Erle Robotics [5], Gym-Gazebo2 by AcutronicRobotics [6], both of which interfaced with Gazebo Classic, and Gym-Ignition by Robotology [7]. However, the latest iteration of Gazebo —Gazebo Sim—currently lacks an easy-to-use toolkit for implementing the modern Gymnasium API.

ARCHITECTURE

Gym-Gazebo-Sim is separated into two primary layers: a gymnasium-gazebo communication layer (green) and the gazebo-ros2 layer (orange) (see Figure 1)

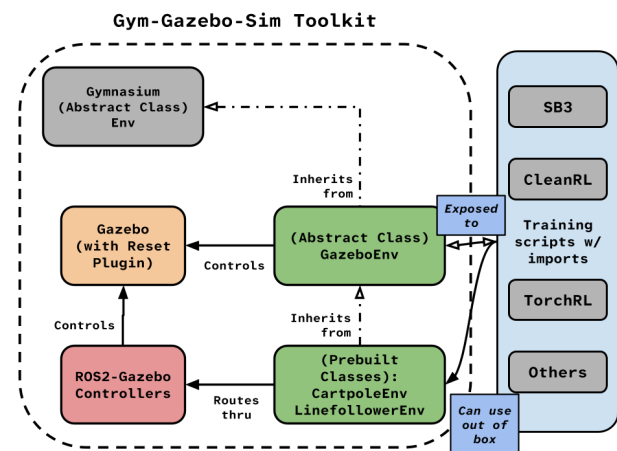


Figure 1: High level architecture of Gym-Gazebo-Sim. The architecture is built around the abstract class GazeboEnv which contains methods to control Gazebo. Prebuilt classes and all other environments inherit from GazeboEnv and can use ROS2 to control agents.

* miti@phas.ubc.ca

Training algorithms that natively interface with Gymnasium can either use prebuilt classes provided by the toolkit (such as `CartpoleEnv` and `LineFollowerEnv`) or inherit from the exposed `GazeboEnv` class, allowing developers to create their own Gymnasium-compatible environments.

The abstract class `GazeboEnv` talks directly to the Gazebo simulator using Gazebo’s internal transport network and gives developers the ability to pause, step, and reset the simulation. The resetting logic is handled by a ROS2 plugin package written in C++ which records the starting position of all non-static entities within the simulator and teleports them back to their starting points when a message is sent to it through a gazebo topic. Moreover, because `GazeboEnv` inherits from the standard Gymnasium `Env` class, custom environments built upon it retain access to all standard Gymnasium methods.

Controller communication between these environments and their respective agents is handled through ROS2 nodes, which publish and subscribe to topics established by Gazebo plugins inside the agent URDF. Using ROS2 gives developers access to the ROS2-Gazebo controller library, alongside the ability to easily manage launch files and worlds within ROS packages, as well as compatibility with sim-to-real development pipelines that rely on ROS2.

A critical component of this architecture is the simulation reset logic which handled by a custom Gazebo plugin written in C++. This plugin resides in its own ROS2 package within the project’s workspace (Figure 2).

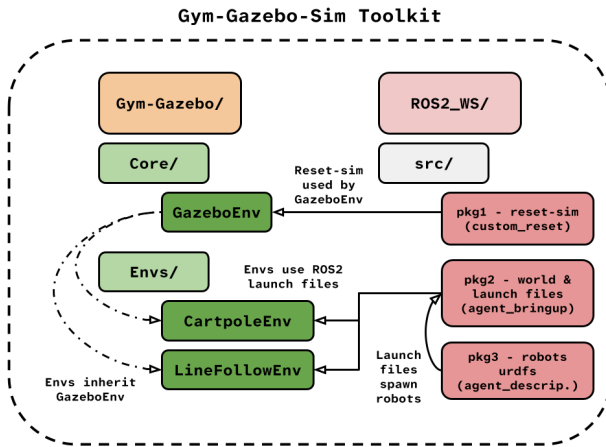


Figure 2: Project file structure. Red boxes labelled ‘pkg’ are ROS2 packages. `GazeboEnv` uses a custom reset plugin stored inside the `custom_reset` package to teleport agents back to their spawn-points. `CartpoleEnv` and `LineFollowerEnv` rely on ROS2 launch and robot description packages to start their worlds and use methods inherited from `GazeboEnv` to handle pausing, stepping, and resetting.

This plugin was developed out of necessity due to Gazebo Sim’s native reset function behaviour. The reset function permanently eliminates all dynamically spawned bodies,

such as `.urdf.xacro` entities spawned via ROS2 launch files. While this limitation can be bypassed by hard-coding agents directly into the world `.sdf` files, the `.sdf` format does not support the importing of `.urdf.xacro` files, forcing developers to abandon the convenient to use `.xacro` architecture.

To solve this, the custom reset plugin searches through all loaded-in entities in Gazebo that have the `<static/>` tag disabled and records their initial spawn positions. When the reset message is received over the Gazebo topic `keyboard/keypress`, the plugin uses Gazebo’s `WorldCommandPose` function to teleport non-static entities back to their spawn points, while zeroing out the velocities and positions the joints of each entity.

Because the simulation reset methods provided in `GazeboEnv` rely entirely on this plugin to function, it is an essential piece of the toolkit. Developers seeking to use `Gym-Gazebo-Sim` to build their own environments must copy the plugin’s ROS2 package into their personal ROS2 workspace and include it within their own world `.sdf` files.

ENVIRONMENTS

As of the release of this white paper, there are only two pre-built environments and agents in `Gym-Gazebo-Sim`: `LineFollowEnv` and `CartpoleEnv`. Each environment starts the Gazebo simulator using a specific launch file specified in the constructor, with different agents for each one.

LineFollowEnv

`LineFollowEnv` is a simple environment that features a 2D recreation of the famous F1 Monza track [8]. The agent is a two-wheeled differential-drive rover that relies on simple computer vision via an onboard camera to detect the track.

Both the action and state spaces in this environment are discrete, making it suitable for Q-learning algorithms. To determine the current state, the camera feed is vertically divided into three equal sections where the state is then defined by identifying which of these sections contains the centroid of the detected line.

The actions that can be sent are as follows:

- Action 0 - Forward 1.0 m/s, Left 1.5 rad/s
- Action 1 - Forward 1.0 m/s
- Action 2 - Forward 1.0 m/s, Right 1.5 rad/s

This environment is registered under the id ‘`LineFollowerEnv-v0`’.

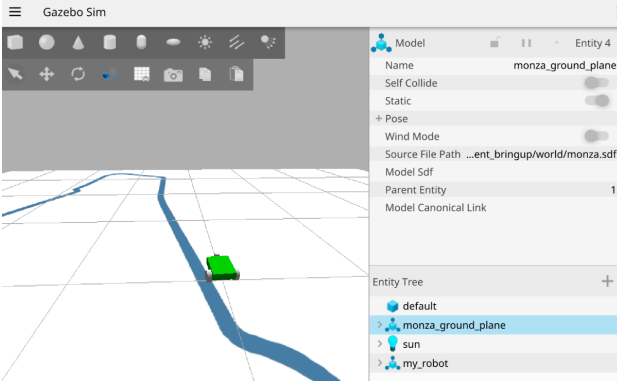


Figure 3: LineFollowEnv-v0 environment in GazeboSim. The green square is the body of the rover and has two white wheels on the rear. The front of the rover glides on a spherical skid.

CartpoleEnv

CartpoleEnv is another simple environment featuring a cart on a rail with a single pendulum balanced on top. The pendulum's axis of rotation is restricted to the y-axis, and the environment returns observations of the cart's position and velocity, as well as the pole's angle from the vertical and its angular velocity.

The actions that can be sent are as follows:

- Action 0: cart_vel + 0.2
- Action 1: cart_vel - 0.2

This environment is registered under the id 'CartpoleEnv-v0'.

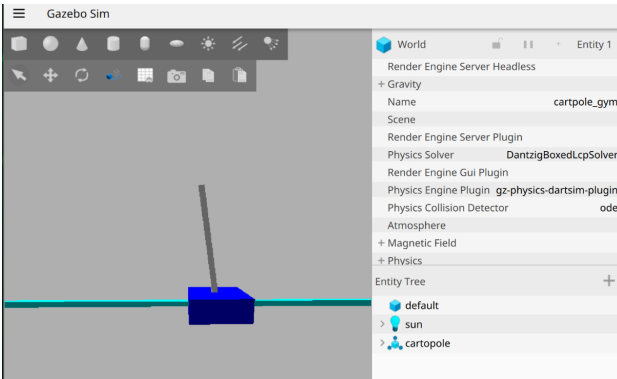


Figure 4: CartpoleEnv-v0 in GazeboSim. The teal component is a fixed rail that the blue cart slides along. The gray pole is free to swing back and forth on the cart, but not to the side.

RESULTS

In this paper, we experimented with two reinforcement learning techniques: Q-Learning and the Cross-Entropy Method (CEM). Q-Learning was evaluated on the LineFollowEnv-v0 environment, while the CEM was applied to CartpoleEnv-v0.

As previously mentioned the toolkit was developed for ENPH 353 a course in the Engineering Physics program at the University of British Columbia. Given the constraints of this educational environment where students often lack the recommended specs for high-fidelity simulators, all reinforcement learning algorithms were trained and evaluated at a real-time factor (RTF) of 1.0. Testing was conducted inside a VirtualBox [9] virtual machine running Xubuntu 24.04, with an allocated 8GB of 3200 MHz RAM and 4 cores of an 11th Gen Intel i7-11800H CPU on a laptop, all without GPU acceleration.

Q-Learning in LineFollowEnv

The small size of both the state and action spaces accelerates the agent's training, as it requires less time to populate the Q-table. Furthermore, the turning actions are designed to carry the exact same forward velocity as the pure forward action. This design choice prevents unwanted behaviors, such as the agent sitting still and indefinitely spinning in place. This forces the agent to maximize its reward by learning to actually follow the line.

The rewards for the agent are as follows:

- Action 0 (Left): +4
- Action 1 (Straight): +5
- Action 2 (Right): +4
- Losing Line (Terminated): -300

Q-Learning is a model-free reinforcement learning algorithm designed to learn the optimal action for any given state by continuously interacting with the environment [10, 11]. The "Q" (which is commonly interpreted to stand for "quality") represents the long-term discounted expected value of taking a specific action in a specific state. The quality for each state-action pair is continuously updated using the Bellman equation, which utilizes a discount factor, γ , to determine how heavily potential future rewards should be weighted when calculating a new value.

The equation for updating the quality of a state-action pair is given by:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

The core of the Q-Learning algorithm implemented in this test is structured as follows:

```
env = gym.make('LineFollowerEnv-v0')
q_table = np.zeros((observation_space.n,
                    action_space.n))

for episode in range(1000):
    state, _ = env.reset()
    done = False

    while not done:
        action = choose_action(state, q_table,
```

```

epsilon, action_space)
    nextState, reward, terminated,
truncated, _ = env.step(action)
    done = terminated or truncated

    current_value = q_table[state, action]
    best_future_value =
np.max(q_table[nextState])
    q_table[state, action] = current_value +
alpha * (reward + gamma * best_future_value -
current_value)

    state = nextState

```

The learning can be tweaked using the following parameters:

- α (**learning rate**): Setting this to 0 means the agent will not learn from new observations when updating the Q-table. Conversely, a high value (e.g., 0.9) means new observations strongly influence the Q-table, rapidly overwriting previous knowledge. This however, could potentially lead to the learning not converging if the frequency and novelty of new observations is too high.
- γ (**discount factor**): Setting this to 0 makes the agent completely myopic, valuing only the immediate reward. A value closer to 1 ensures that the agent prioritizes long-term expected rewards.
- ϵ (**exploration rate**): Used to control the randomness of decision-making; setting a high value such as 0.8 ensures 80% of actions are completely stochastic. A common and highly effective technique is epsilon decay, where the agent begins in an exploration phase (taking random actions to discover the environment) and gradually transitions into a greedy exploitation phase (relying entirely on the Q-table for optimal actions).

For this test, the following parameters were selected:

- $\alpha = 0.1$
- $\gamma = 0.9$
- $\epsilon = 0.9$, epsilon decay = 0.99, and $\epsilon_{\min} = 0.01$

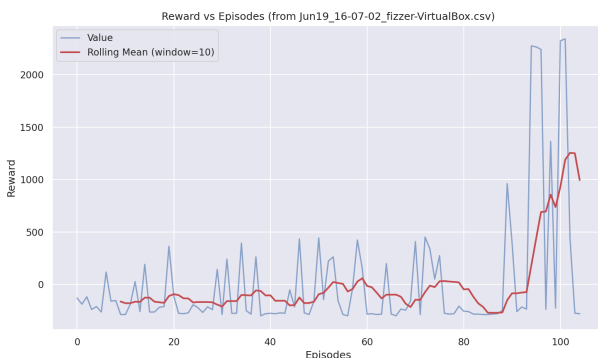


Figure 5: Cumulated reward per episode after 105 episodes. A reward over 1,000 indicates the car can follow the line over most of the track.

The Q-learning algorithm converges very quickly in this

environment. A total reward of ~1,100 or higher for a single episode typically indicates a near-complete traversal of the Monza track which, as seen in the data, is achieved in roughly 100 episodes.

Cross Entropy Learning on CartpoleEnv

The CEM is a model-free, policy-based reinforcement learning algorithm [11]. Instead of calculating the expected value of future rewards (like Q-Learning), CEM directly optimizes the agent's behavior through an evolutionary optimization process.

The agent plays out multiple episodes using its current neural network to predict action probabilities. Once a batch of episodes is completed, they are ranked by their total cumulative reward. Top-performing episodes are then isolated, and the network is then trained using supervised learning (specifically Cross-Entropy Loss) to map the observations from those elite episodes to the actions that were taken.

The CartpoleEnv-v0 environment has a discrete action space and continuous observation space. The observation space consists of the cart's position, velocity, and the pole's angle and angular velocity, all of which are continuous floating point numbers. Like the line-following agent, this agent learns to maximize the cumulative reward for each episode.

The rewards for the agent (the cart) is as follows:

- Terminated ($|x| > 15\text{m}$, $|\theta| > 12^\circ$): 0
- Not terminated: +1

(Note): x refers to the horizontal distance of the cart from the origin along the rail. θ is the angle of the pole from the vertical axis.

The simplified algorithm was derived from “Deep Reinforcement Learning hands on” [11]. It was implemented in this test is as shown:

```

net = NeuralNet(obs_size, hidden_size, n_action)
objective = nn.CrossEntropyLoss()
optimizer = optim.Adam(params=net.parameters(),
lr=0.01)

for iter_no, batch in
enumerate(generate_batches(env, net,
batch_size)):
    # Isolate the elite episodes based on the
    percentile cutoff
    obs_tensor, action_tensor, reward_cutoff,
reward_mean = filter_batches(batch, percentile)

    optimizer.zero_grad()

    # Action probabilities predicted by the
    Neural Network
    act_logits_tensor = net(obs_tensor)

    # Calculate loss between predicted actions
    and the actual actions taken in elite episodes

```

```

    loss_tensor = objective(act_logits_tensor,
action_tensor)

    # Backpropagate and update network weights
    loss_tensor.backward()
    optimizer.step()

    # Stop training once the mean reward of a
    batch hits the target
    if reward_mean > reward_threshold:
        print("Solved!")
        break

```

The neural network itself is a standard Multi-Layer Perceptron (MLP) featuring a single hidden layer with a ReLU activation function, outputting raw logits that are converted into a probability distribution via a Softmax function.

The learning behavior of this algorithm can be tweaked using the following parameters:

- **batch_size**: The number of episodes the agent plays through before performing a network update. A larger batch size provides more stable data but takes longer to compute.
- **percentile**: The threshold used to filter “elite” episodes. Setting this to 70 means the algorithm discards the bottom 70 of episodes and only trains the network on the top 30 of highest-rewarding trajectories.
- **hidden_size**: The number of neurons in the network’s hidden layer, dictating the model’s capacity to learn complex state-to-action mappings.
- **lr (learning rate)**: The step size used by the Adam optimizer when updating the network’s weights during backpropagation.

For this test, the following parameters were selected:

- batch_size=16
- percentile=70
- hidden_size=128
- lr=0.01

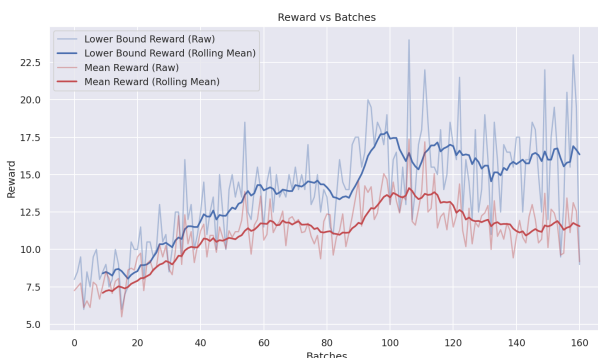


Figure 6: Cumulated reward per batch after 160 episodes. Despite not training long enough to converge on a completely successful result, the mean reward for each batch of episodes increases as more iterations are run.

The Cross-Entropy method algorithm converged much more slowly than in the Q-Learning example. This is likely due to the fact that optimization occurs every batch (16 episodes) instead of every episode. However, there was still a noticeable increase in the mean reward and lower bound reward of each episode. The lower bound reward refers to the reward cutoff within a batch, below which the episodes are discarded instead of being processed for training.

CONCLUSION & FUTURE PATH

The Gym-Gazebo-Sim toolkit is capable of running simple training algorithms on the single-agent pre-built environments at 1 RTF given the strict hardware requirements placed upon it. However, there are still some areas of improvement for the toolkit:

- Implementing more environments for benchmarking (LiDAR drones and turtle bot).
- Creating a global installer for the custom reset plugin.
- Creating ready-to-go gymnasium observation and action spaces for ROS2 sensor topics.
- Multi-environment and multi-agent training.

AUTHOR CONTRIBUTIONS

- Conceptualization: M. Isbasescu and T. Momose. Supervision: M. Isbasescu. Design and implementation: T. Momose. Refactoring and optimization: M. Isbasescu. Writing original draft: T. Momose; Reviewing and editing: M. Isbasescu and T. Momose.

REFERENCES

- [1] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments”, 2024. <https://arxiv.org/abs/2407.17032>
- [2] Gazebo-sim, <https://gazebo-sim.org/home>
- [3] M. Lee, “1.6 history of reinforcement learning”, in *Reinforcement Learning: An Introduction*, Cambridge, MA, USA: The MIT Press, 2005. <http://incompleteideas.net/book/ebook/node12.html>
- [4] OpenAI, “OpenAI Gym”, <https://github.com/openai/gym>
- [5] I. Zamora, N. G. Lopez, V. M. Vilches, and A. H. Cordero, “Extending the Openai gym for robotics: A toolkit for reinforcement learning using ROS and gazebo”, 2016. <https://arxiv.org/abs/1608.05742>
- [6] N. G. Lopez *et al.*, “Gym-gazebo2, a toolkit for reinforcement learning using ROS 2 and gazebo”, 2019. <https://arxiv.org/abs/1608.05742>
- [7] D. Ferigo, S. Traversaro, G. Metta, and D. Pucci, “Gym-ignition: Reproducible robotic simulations for reinforcement learning”, 2019. [doi:10.1109/SII46433.2020.9025951](https://doi.org/10.1109/SII46433.2020.9025951)
- [8] Autodromo Nazionale Monza, https://en.wikipedia.org/wiki/Monza_Circuit
- [9] Virtual Box, <https://www.virtualbox.org/>

- [10] Q-Learning in Reinforcement Learning, <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>.
- [11] M. Lapan, in *Deep Reinforcement Learning Hands-On*, Birmingham, UK: Packt, 2000, pp. 143–156180–200.